

Database Systems and Laboratory

Relational model

Nicola Saccomanno

`nicola.saccomanno@uniud.it`

Bachelor degrees in Internet of things, Big Data, Machine Learning, & Computer Science
DMIF, University of Udine

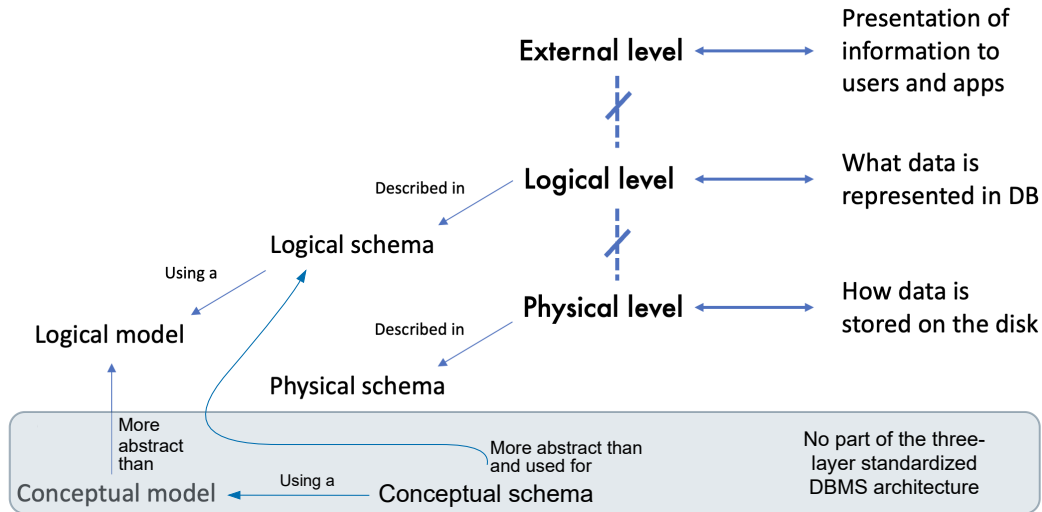
A.Y. 2025/2026

Database design

The process of designing a database is typically articulated into different phases:

1. Brainstorming meetings with IT personnel and all interested stakeholders
 - ▶ Collection of requirements
 - ▶ Design of a conceptual model (e.g., E-R model)
 - ▶ The E-R model has a specific notation, the *E-R diagram*, that helps all involved parts to discuss about the future database
2. Translation of the conceptual model into a logical model
 - ▶ Typically, a number of transformation is applied, followed by a set of translation rules
 - ▶ At this stage, a specific DBMS technology must be chosen (e.g., relational DB)
3. Addition to the logical model of details regarding the physical, low-level aspects (e.g., usage of indexes)
 - ▶ The physical schema is thus obtained

Recap: Levels, schemas, and models



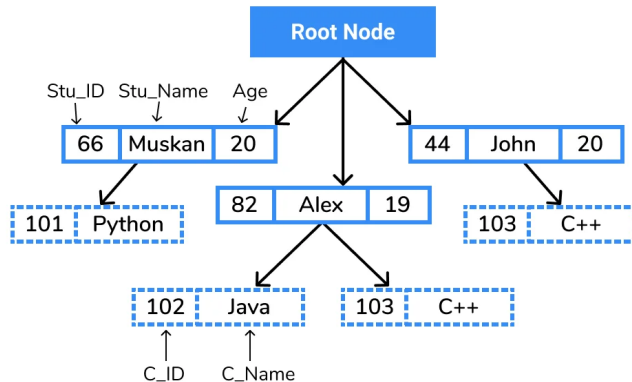
Some context

- ▶ In the previous part of the course we saw how to develop the conceptual schema of a database
- ▶ The next step is that of choosing a suitable data model to represent the information at the logical level
- ▶ Then, the conceptual model gets translated into such a logical model, typically following a specific set of rules
- ▶ In this set of slides, we will start presenting the **relational model**

Logical data models

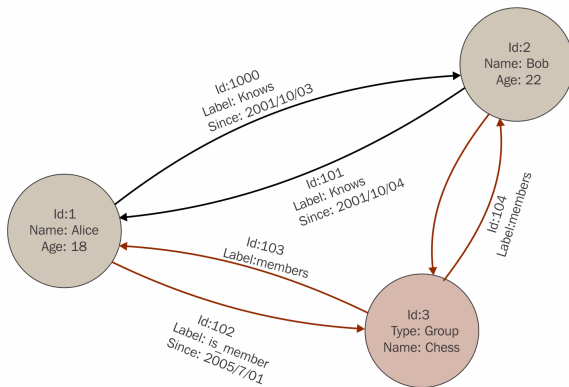
- ▶ *Logical data models* are abstract: they allow to describe data irrespective of how they will be physically stored → Aim: physical data independence
- ▶ Still, they reflect a specific organization of the data (using tables, graphs, etc)
- ▶ There exist several logical data models
 - ▶ The **relational data model**, which considers records and relations (rows and tables)
 - ▶ Graph data model, that deals with nodes and vertices
 - ▶ Object-based data models
 - ▶ Semistructured data models (XML)
 - ▶ Older models: Hierarchical model, Network model
- ▶ Hierarchical and network closer to physical structures, relational higher level
 - in the relational model we have only values: even references between data in different sets (relations) are represented by means of values
 - in the hierarchical and network model there are explicit references (pointers)

Example: Hierarchical data model



Records physically point to other records; only “parent-child” relationships are allowed

Example: Graph data model



Records physically point to other records; more flexible structure for the nodes and the relationships

The relational model

- ▶ Proposed by E. F. Codd (IBM Research) in 1970 in order to support data independence
 - ▶ "A Relational Model for Large Shared Data Banks," Communications of the ACM, June 1970
- ▶ Made available in commercial DBMSs in 1981 (it is not easy to implement data independence efficiently and reliably!)
- ▶ It is based on (a variant of) the mathematical notion of **relation**
- ▶ Relations are naturally represented by means of tables

Informal Definitions

- ▶ Informally, a **relation** looks like a **table** of values.
- ▶ A relation typically contains a **set of rows**.
- ▶ The data elements in each **row** represent certain facts that correspond to a real-world **entity** or **relationship**
 - In the formal model, rows are called **tuples**
- ▶ Each **column** has a column header that gives an indication of the meaning of the data items in that column
 - In the formal model, the column header is called an **attribute name** (or just **attribute**)
- ▶ Each row has a value of a data item/column (or set of items/columns) that **uniquely identifies** that row in the table
 - In the formal model, the **key**

Example of a Relation

Relation Name

STUDENT

Attributes

Tuples

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	NULL	19	3.21
Chung-cha Kim	381-62-1245	375-4409	125 Kirby Road	NULL	18	2.89
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	749-1253	25	3.53
Rohan Panchal	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	NULL	19	3.25

Mathematical relations

- ▶ $D_1, D_2, \dots, D_n$ (n sets, not necessarily distinct)
 - ▶ A domain has a logical definition, a data-type or a format defined for it
- ▶ **cartesian product** $D_1 \times D_2 \times \dots \times D_n$:
 - the set of all (ordered) n -tuples $(d_1, d_2, \dots, d_n)$ such that $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$
- ▶ a **mathematical relation** on $D_1, D_2, \dots, D_n$:
 - a subset of the cartesian product $D_1 \times D_2 \times \dots \times D_n$
- ▶ $D_1, D_2, \dots, D_n$ are the **domains** of the relation
- ▶ n is the **degree** of the relation
- ▶ the number of n -tuples is the **cardinality** of the relation; in practice, it is always finite

Mathematical relations, properties

- ▶ A mathematical relation is a **set of ordered** n-tuples $(d_1, d_2, \dots, d_n)$, i.e., a collection where each element is an ordered list of exactly n values, such that $d_1 \in D_1, d_2 \in D_2, \dots, d_n \in D_n$
- ▶ a set, so:
 - there is no ordering between n-tuples (even though they appear to be in the tabular form!)
 - the n-tuples are distinct from one another
- ▶ the n-tuples are **ordered**: the i-th value come from the i-th domain $\rightarrow$ so there is an ordering among the domains

Mathematical relation, example

$$games \subseteq string \times string \times integer \times integer$$

Juve	Lazio	3	1
Lazio	Milan	2	0
Juve	Roma	1	2
Roma	Milan	0	1

- ▶ Each of the domains has two **roles**, distinguished by means of position
- ▶ The structure is **positional**

Relations in the relational data model

- ▶ We would like to have a **non-positional** structure
- ▶ We associate a unique name (**attribute**) with each domain; it “describes” the role of the domain in the specific relation
- ▶ In the tabular representation, attributes are used as column headings

HomeTeam	VisitingTeam	HomeGoals	VisitorGoals
Juve	Lazio	3	1
Lazio	Milan	2	0
Juve	Roma	1	2
Roma	Milan	0	1

Formalizing

- ▶ Let $\mathcal{X}$ be the set of attributes and $\mathcal{D}$ the set of all domains
- ▶ The correspondence between attributes and domains:

$$\text{dom} : \mathcal{X} \rightarrow \mathcal{D}$$

- ▶ A tuple on $\mathcal{X}$ is a function that associates with each $A \in \mathcal{X}$ a value from the domain $\text{dom}(A)$
- ▶ A **relation** on $\mathcal{X}$ is a set of tuples on $\mathcal{X}$

Notation

- ▶ $t[A]$ (or $t.A$) denotes the value on A of a tuple t
- ▶ In the example, if t is the tuple represented by the first row in the table

$$t[\text{VisitingTeam}] = \text{Lazio}$$

- ▶ The same notation is extended to sets of attributes, thus denoting tuples:

$$t[\text{VisitingTeam}, \text{VisitorGoals}]$$

is a tuple on two attributes

Attribute of relations

A note on composite and multivalued information

- ▶ Attribute values are required to be **atomic**, that is, indivisible (first normal form property)
- ▶ This means that we cannot have, in the records, composite fields such as: *address*, made of *city*, *street*, *number*
- ▶ Thus, we have two choices:
 - ▶ Join together the composing fields into a single attribute
 - ▶ Store only the composing fields, each as a separate attribute
- ▶ Also, there cannot be multivalued attributes: they should be represented by a table on their own

The relational model is “value-based”

Students

RegNum	Surname	FirstName	BirthDate
6554	Rossi	Mario	5/12/1978
8765	Neri	Paolo	3/11/1976
9283	Verdi	Luisa	12/11/1979
3456	Rossi	Maria	1/2/1978

- **References** between data in different relations are represented **by means of values** of the domains

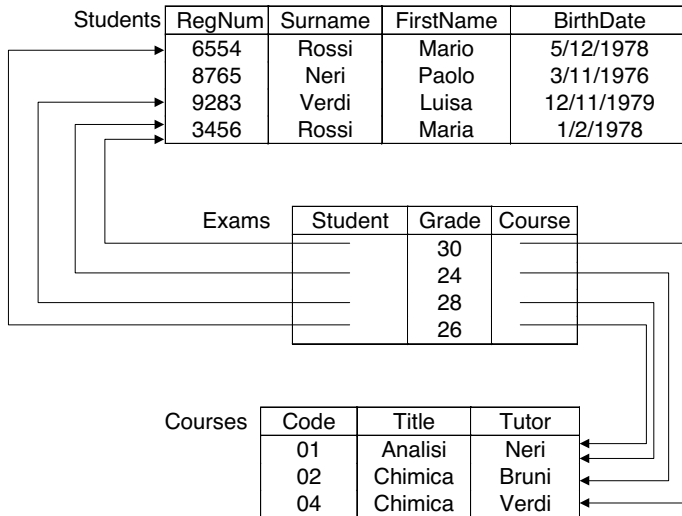
Exams

Student	Grade	Course
3456	30	04
3456	24	02
9283	28	01
6554	26	01

Courses

Code	Title	Tutor
01	Analisi	Neri
02	Chimica	Bruni
04	Chimica	Verdi

The relational model is “value-based”



Advantages of a value-based structure

- ▶ **Independence** of physical structures
- ▶ Only information that is relevant from the **application** point of view
- ▶ Easy **transferability** of data between systems

Notes:

- ▶ pointers usually exist at the physical level, but they are not visible at the logical level
- ▶ object-identifiers in object databases show some features of pointers, at a higher level of abstraction

Definitions

Relation schema:

a name (of the relation) R with a set of attributes $A_1, \dots, A_n$
 $R(A_1, \dots, A_n)$

Database schema:

a set of relation schemas with different names
 $\mathcal{R} = \{R_1(X_1), \dots, R_n(X_n)\}$

Relation (instance) on a relation schema $R(X)$:

set r of tuples on X (also $r(R)$)

Database (instance) on a database schema $\mathcal{R} = \{R_1(X_1), \dots, R_n(X_n)\}$:

set of relations $\mathbf{r} = \{r_1, \dots, r_n\}$ (with r_i relation on R_i)

Examples

Students

RegNum	Surname	FirstName	BirthDate
6554	Rossi	Mario	5/12/1978
8765	Neri	Paolo	3/11/1976
9283	Verdi	Luisa	12/11/1979
3456	Rossi	Maria	1/2/1978

- ▶ Relations on a single attribute are also admissible

Workers

RegNum
6554
8765

Definition Summary

Informal Terms	Formal Terms
Table	Relation
Column Header	Attribute
All possible Column Values	Domain
Row	Tuple
Table Definition	Schema of a Relation
Populated Table	Instance of the Relation

Incomplete information

- ▶ The relational model impose a rigid structure to data:
 - information is represented by means of tuples
 - tuples have to conform to relation schemas
- ▶ In practice, the **available data need not conform to the required formats**
- ▶ *Example:*
 - ▶ (County towns have government offices, other cities do not)
 - ▶ Florence is a county town; so it has a government office, but we do not know its address
 - ▶ Tivoli is not a county town; so it has no government office
 - ▶ Prato has recently become a county town; has the government office been established? We don't know

City	GovtAddress
Rome	Via IV novembre
Florence	
Tivoli	
Prato	

Incomplete information: solutions?

- ▶ (Despite what often happens) we should **not use domain values** (zero, 99, empty string, etc.) to represent lack of information:
 - there need not be “unused” values
 - “unused” values could become meaningful
 - in programs, we should be able to distinguish between actual values and placeholders (for example: calculate the average age of a set of people, where 0 is used for unknown ages!)
- ▶ A simple but effective technique is used:
 - **null value**: a special value (not a value of the domain) denotes the absence of a domain value
 - A tuple on $\mathcal{X}$ is a function that associates with each $A \in \mathcal{X}$ a value from the domain $\text{dom}(A)$ or NULL
- ▶ We could (and often should) put restrictions on the presence of null values in tuples (we will see later)

Types of null value

- ▶ (At least) three types of null value
 - **unknown value**: there is a domain value, but it is not known (Florence)
 - **non-existent value**: the attribute is not applicable for the tuple (Tivoli)
 - **no-information value**: we don't know whether a value exists or not (Prato); this is the disjunction (logical or) of the other two
- ▶ **DBMSs** do not distinguish between the types: they implicitly **adopt the no-information value**

How to prevent meaningless database instance?

Exams

RegNum	Name	Course	Grade	Honours
6554	Rossi	B01	K	honours honours
8765	Neri	B03	C	
3456	Bruni	B04	B	
3456	Verdi	B03	A	

Courses

Code	Title
B01	Physics
B02	Calculus
B03	Chemistry

- ▶ grades are between A and F
- ▶ honours can be awarded only if grade is A
- ▶ different students must have different registration numbers
- ▶ exams must refer to existing courses

Integrity constraints

- ▶ In order to **preserve the consistency and quality** of information stored in the database, it is possible (and highly suggested) to define integrity constraints **limiting the data** that can be stored within the tables
- ▶ **Intra-relational** constraints:
 - ▶ Not-null constraint
 - ▶ Uniqueness constraint
 - ▶ Primary key
 - ▶ General tuple-level constraints
- ▶ **Inter-relational** constraints:
 - ▶ Foreign key
- ▶ Some constraint are implicitly defined by the data model
- ▶ It is not the case that all properties can be described by means of integrity constraints → **some constraints beyond the expressive power of the model**

Tuple constraint

- ▶ Express **conditions on the values of each tuple**, independently of other tuples
- ▶ A possible syntax: boolean expressions with atoms that compare attributes, constants or expressions over them
- ▶ **Domain constraint**: a tuple constraint that involve a single attribute
- ▶ A domain constraint

$$(\text{Grade} \geq \text{"A"}) \text{ AND } (\text{Grade} \leq \text{"F"})$$

- ▶ A tuple constraint

$$(\text{NOT } (\text{Honours} = \text{"honours"})) \text{ OR } (\text{Grade} = \text{"A"})$$

- ▶ A tuple constraint example on the schema $P(\text{Date}, \text{Net}, \text{Amount}, \text{Deductions})$:

$$\text{Net} = \text{Amount} - \text{Deductions}$$

Not-null constraint

- ▶ Defined **over a column**, it **forbids null** values
- ▶ In the following example, null values are allowed on column *state*
- ▶ To model a situation in which every customer is always associated to a proper *state*, it is possible to define a not-null constraint over the associated column
 - ▶ $VNN : \{state\}$
- ▶ As a result, rows with null values for *state* (last three in the example) will be rejected (note how this changes the semantics of the modeled domain)

	customername	state	country
▶	Australian Collectors, Co.	Victoria	Australia
	Anna's Decorations, Ltd	NSW	Australia
	Souvenirs And Things Co.	NSW	Australia
	Australian Gift Network, Co	Queensland	Australia
	Australian Collectables, Ltd	Victoria	Australia
	Salzburg Collectables	NULL	Austria
	Mini Auto Werke	NULL	Austria
	Petit Auto	NULL	Belgium

Uniqueness constraint

- ▶ The uniqueness constraint can be defined **over one or more columns**
- ▶ It forces the values stored in a column or group of columns **to be unique among the table rows**
- ▶ E.g., in a table storing information on hotels, we may forbid the presence of multiple hotels that are in the same city and that have the same name
- ▶ To do that, we can define a uniqueness constraint over the column group `UNIQUE: {(Nome_albergo, Citta')}`
- ▶ Note how this is different than defining two uniqueness constraints, over `Nome_albergo` and over `Citta'`, i.e., `UNIQUE: {(Nome_albergo), (Citta')}`

Partiva IVA	Nome albergo	Città	Stelle	Numero camere
1053362646	Ritz	Roma	5	205
1053362600	Ritz	Milano	4	215
1233362688	Da Mimmo	Napoli	3	80
1053369902	Friuli	Udine	3	50
1325239931	Friuli	Udine	2	45

Primary key constraint

Intuition

- ▶ Some attributes play a fundamental role
- ▶ Knowing their value, it is possible to **uniquely identify a tuple inside a table**
- ▶ For instance:
 - ▶ We can expect *SSN* to uniquely identify a row in a table such as *Person(SSN, Name, Surname, Nationality)*
 - ▶ We can expect *VAT_code* to uniquely identify a row in a table such as *Hotel(VAT_code, Name, Stars, City)*
 - ▶ Note how, in the same table *Hotel*, the pair (*Name, City*) may or may not uniquely identify a hotel; it really depends on the kind of reality that we want to model

Primary key constraint

Another example

- ▶ Consider the following table, that records information regarding drugs *Drug*(*COde*, *Producer*, *Name*, *Posology*)
- ▶ We assume each drug to be uniquely identified by its *Code*
- ▶ Also, we assume that there may not be two different drugs with same *Name* and *Producer*
- ▶ Thus, there are two ways by which we can uniquely identify a row in the table

Code	Producer	Name	Posology
AX124	Bayer	Aspirin	A
AX127	Menarini	VIVIN C	A
AB123	Angelini	Moment 200	B
AB234	Angelini	Tachipirina 500	B
KL253	SANOFI	Enterogermina	C

Figure: Relation “Drug”

Primary key constraint

Definition

- ▶ Given $R(X)$, let $K \subseteq X$ (K is a subset of the attributes of rel. schema R)
- ▶ K is a **superkey** of R if the values for K are sufficient to identify in a unique way any tuple in all possible instantiations of R
 - ▶ E.g., $\{Code\}$, $\{Code, Posology\}$, and $\{Name, Producer\}$ are all superkeys of *Drug*
- ▶ Superkey K is a **candidate key** if K is minimal
 - ▶ Minimal: if we remove an attribute from K , then it is not a superkey anymore
 - ▶ E.g., $\{Code\}$, as well as $\{Name, Producer\}$ is a candidate key for *Drug*
- ▶ One of the candidate keys is chosen as the **primary key**
 - ▶ Typically the smallest one
 - ▶ We typically highlight the primary key underlying it in the schema, e.g.,
Drug(Code, *Producer*, *Name*, *Posology*) or *Drug*(*Code*, *Producer*, *Name*, *Posology*)

Existence and importance of keys

- ▶ Relations are sets; therefore each relation is composed of distinct tuples: **the whole set of attributes is always superkey**;
- ▶ so **each relation has always a superkey**; since the set of attributes is finite, **each relation schema has at least a candidate key**:
 - the whole set is either a candidate key
 - or it contains a (smaller superkey), and for it we can repeat the argument, over a smaller set
- ▶ The existence of keys guarantees that each piece of data in the database can be accessed
- ▶ Keys are the major feature that allows us to say that the relational model is “value-based”

Keys and null values

- ▶ If there are nulls, keys do not work that well
 - they do not guarantee unique identification
 - they do not allow to establish correspondences between data in different relations

RegNum	Surname	FirstName	BirthDate	DegreeProg
NULL	Smith	John	NULL	Computing
587614	Smith	Lucy	01/05/61	Engineering
934856	Black	Lucy	NULL	NULL
NULL	Black	Lucy	05/03/58	Engineering

- ▶ How do we access the first tuple?
- ▶ Are the third and fourth tuple the same?
- ▶ **Entity Integrity:** the primary key attributes K of each relation schema R in a database schema $\mathcal{R}$ cannot have null values in any tuple $t[K] \in r(R)$.

Primary key constraint

Notes

- ▶ The **primary key** plays a fundamental role in relations; if well defined, with respect to the considered domain, it **prevents** the presence of **duplicate** and possibly **inconsistent data**
- ▶ The primary key allows one to **identify** the data contained in a table, and it is also through primary key values that the **relationships between tables** are built
- ▶ The primary key constraint **naturally implies** the presence of two further constraints:
 - ▶ **Entity integrity**: attributes belonging to the primary key cannot be null
 - ▶ **Uniqueness**: since by definition in a table there cannot be two different rows having the same values for the primary key attributes

Primary key constraint

Notes

- ▶ Observe how the definition of candidate keys strictly depends on domain knowledge
- ▶ For instance, in the following table, even if there are no two students with the same name and surname, it is not sensible to consider such two attributes as a candidate key
- ▶ The two innate candidate keys are *sid* and *login*; thus, one of them will be set a primary key constraint, and the other one a uniqueness constraint

sid	fname	lname	login	age	GPA
C123456	John	Smith	smithx02@ece.abc.edu	19	3.3
C234561	Karen	Johns	johnsx05@cs.abc.edu	18	3.0
C345612	Mary	Anderson	anderx10@math.abc.edu	20	3.5
C456123	Helen	Henderson	hendex05@ece.abc.edu	18	2.9
C561234	John	Smith	smithx99@cs.abc.edu	19	3.8
C612345	Aidan	Cocke	cockex35@ece.abc.edu	18	3.3

Primary keys

Exercise

- ▶ A candidate key of relation *Book*(*ISBN*, *author*, *year*, *editor*, *genre*, *num_pages*) is ISBN
 - ▶ And if we further assume that an author can write at most one book in a given year?
 - ▶ And if we further assume that an author can write at most one book in a given year for a given editor?

Primary keys: do we always have a natural one?

- ▶ In most cases we do have reasonable/natural primary keys already in the data
- ▶ In other cases we don't:
 - we need to introduce new, artificial attributes (identifying “codes”)
 - these are called artificial key or surrogate key
- ▶ Note that most of the “obvious” codes we have now (social security number, student number, area code, ...) were introduced (possibly before the adoption of databases) with the same goal: unambiguous identification of objects

Foreign key constraint

- ▶ A foreign key (aka referential integrity) constraint is defined between two relations, and it allows us to “link” them → specify a relationship among tuples
- ▶ Intuitively, it states that the values appearing in one relation (**referencing relation**) must also appear in the other relation (**referenced relation**)
 - ▶ E.g., given *instructor*(ID, name, dept_name, salary) and *department*(name, building, budget) we can define the foreign key constraint $\{instructor.dept_name\} \rightarrow \{department.name\}$ for the *instructor* (i.e., the referencing) relation
 - ▶ Then, every value in *instructor.dept_name* must be present in *department.name* (or be *null*, if allowed to) → $\{instructor.dept_name\} \subseteq \{department.name\}$
- ▶ A foreign key can be composed of multiple attributes
 - ▶ *director*(SSN, hotel_name, hotel_city, salary)
 - ▶ *hotel*(name, city, stars)
 - ▶ FK: $\{director.hotel_name, director.hotel_city\} \rightarrow \{hotel.name, hotel.city\}$

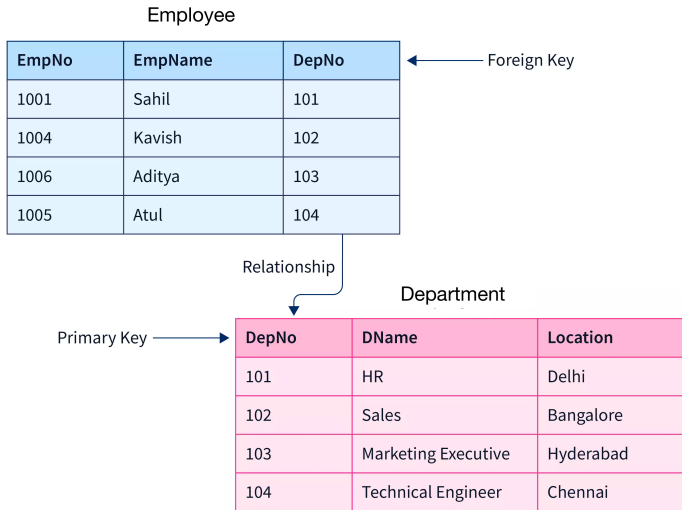
Foreign key constraint

Notes

- ▶ The **attributes referenced by the foreign key must form a superkey in their relation** (typically, the primary key)
 - ▶ This is quite natural, for otherwise it would not be clear which row we are referring to through the foreign key
- ▶ The number and domain of the attributes must be the same in the two relations
 - ▶ Note that foreign key constraint represent an inclusion
 - ▶ Given the foreign key attributes FK for the relation schema R , and primary key (or superkey) attributes PK for the relation schema S , given the relation instances $r(R)$ and $s(S)$ for each tuple $t_r \in r$ s.t. $t_r[FK] \neq NULL$, there must exist a tuple $t_s \in s$ s.t. $t_r[FK] = t_s[PK]$
- ▶ A foreign key can also be defined over the same table, e.g.:
 - ▶ $employee(SSN, name, surname, salary, supervisor)$
 - ▶ FK: $\{employee.supervisor\} \rightarrow \{employee.SSN\}$

Foreign key constraint

Graphical example for FK: $\{employee.DepNo\} \rightarrow \{Department.DepNo\}$



Foreign key constraint

Handling row deletions in the referenced relation

- ▶ What should be done when a referenced value is deleted?
- ▶ Several alternatives:
 - ▶ Refuse deletion
 - ▶ Delete cascade
 - ▶ Set a default/null value

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
22222	Einstein	Physics	95000
12121	Wu	Finance	90000
32343	El Said	History	60000
45565	Katz	Comp. Sci.	75000
98345	Kim	Elec. Eng.	80000
76766	Crick	Biology	72000
10101	Srinivasan	Comp. Sci.	65000
58583	Califieri	History	62000
83821	Brandt	Comp. Sci.	92000
15151	Mozart	Music	40000
33456	Gold	Physics	87000
76543	Singh	Finance	80000

(a) The *instructor* table

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Biology	Watson	90000
Elec. Eng.	Taylor	85000
Music	Packard	80000
Finance	Painter	120000
History	Painter	50000
Physics	Watson	70000

(b) The *department* table

Exercise 1

Let us consider the following relational schema about authors and books

AUTHOR(*name*, *surname*, *birth_date*, *nationality*)

BOOK(*title*, *genre*, *num_pages*, *publication_year*)

WRITES(*author_name*, *author_surname*, *book_title*, *book_year*)

- ▶ Let us assume each author to be uniquely identified by his/her name and surname, and further described by his/her birth date and nationality.
- ▶ We also assume that there may be more than one book with the same title, however, given a year, that cannot be two different books having the same title.
- ▶ Finally, consider that a book may be written by more than one author, and an author can write, in general, more than one book.

Define the primary keys, uniqueness constraints (if any), and foreign keys (if any). Possibly justify your choices by providing some assumptions.

Exercise 2

Let us consider the following relational schema about speakers and conferences

SPEAKER(*name*, *address*, *telephone*, *email*)

ATTENDANCE(*speaker_name*, *conference_name*, *conference_year*)

CONFERENCE_EDITION(*name*, *year*, *city*)

- ▶ Let us assume each speaker to be uniquely identified by his/her name, and further characterized by his/her address, phone number and personal email.
- ▶ Assume that each edition of a conference is uniquely identified by the name of the conference and the year in which the specific edition was held (for example, ICDM 2019 and ICDM 2020 identify two different editions of the ICDM conference).
- ▶ Also, consider that a given conference edition (e.g., ICDM 2024) cannot be hosted by multiple cities. Yet, a city can host at most one conference per year.
- ▶ Finally, assume that at each conference edition there can be several speakers and that the same speaker, generally, can participate to multiple conference editions.

Define the primary keys, uniqueness constraints (if any), and foreign keys (if any).

Possibly justify your choices by providing some assumptions.

Exercise 3

Let us consider the following relational schema about musicians and bands.

MUSICIAN(*name*, *surname*, *art_name*, *nationality*, *band*)

BAND(*name*, *foundation_year*, *genre*, *frontman*)

- ▶ Let us assume each musician to be uniquely identified by his/her art name.
- ▶ Also, assume each band to be uniquely identified by its name.
- ▶ A musician can be part of at most one band; and, a band can be composed of multiple musicians.
- ▶ Finally, each band has a frontman, who is a musician himself/herself.

Define the primary keys, uniqueness constraints (if any), and foreign keys (if any). Possibly justify your choices by providing some assumptions.

References

Material for these slides is partly based on:

- ▶ Atzeni et al., *Database Systems: Concepts, Languages and Architectures*, Chapter 2: "The Relational Model" (book chapter and accompanying slides)
- ▶ Elmasri and Navathe, *Fundamentals of Database Systems*, 7th Edition, Chapter 5: "The Relational Data Model and Relational Database Constraints" (book chapter and accompanying slides)